Lecture 10

Red-Black Trees: Deletion (contd.), Augmenting Data Structures

Source: Introduction to Algorithms, CLRS

Cases for Deletion

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red. → *Case 1 will convert to Case 2, 3, or 4*
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

Case 2 will either terminate or move x towards the root

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

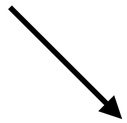
- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

Case 3 will convert to Case 4

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.



Case 4 will terminate

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

What if x or its sibling w is NIL?

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x is NIL or x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x's sibling w is red.
- **Case 2:** x is NIL or x's sibling w is black, and both of w's children are **black**.
- **Case 3:** x's sibling w is black, and w's left child is red, and w's right child is **black**.
- **Case 4:** x's sibling w is black, and w's right child is red.

Idea: Give x an **extra black colour** that it will lose without violating RB-tree properties.

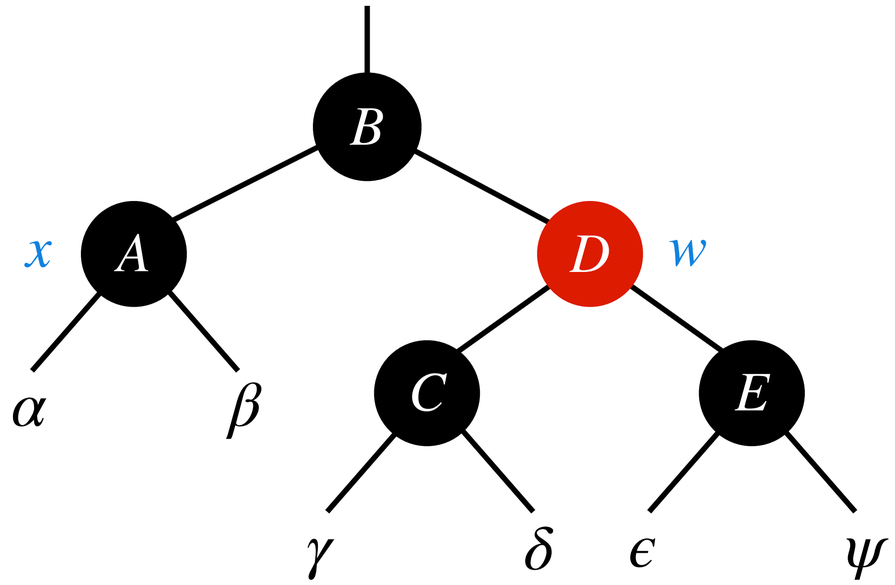
Cases for Deletion

Cases for Deletion

Case 1: x's sibling w is red.

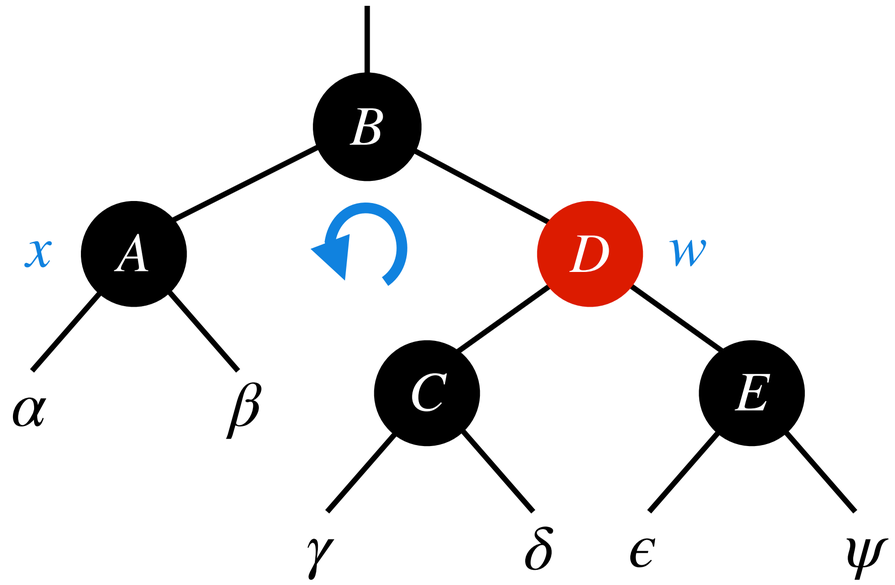
Cases for Deletion

Case 1: x's sibling w is red.



Cases for Deletion

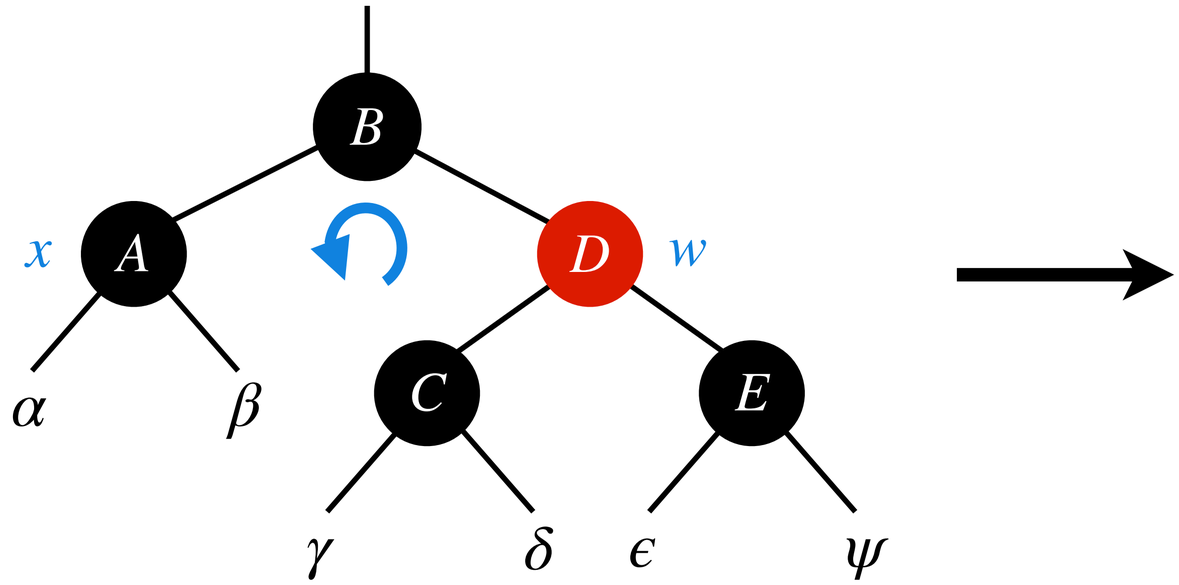
Case 1: x's sibling w is red.



Handling: Switch colours of w and $x\,.\,p$ and perform a left rotation at $x\,.\,p$.

Cases for Deletion

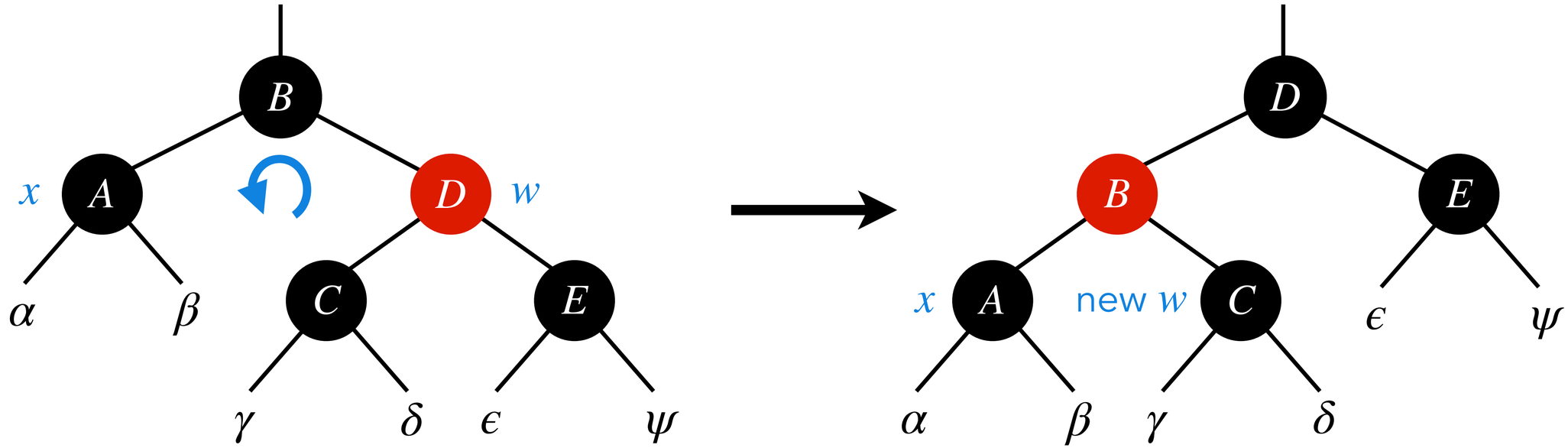
Case 1: x's sibling w is red.



Handling: Switch colours of w and $x\,.\,p$ and perform a left rotation at $x\,.\,p$.

Cases for Deletion

Case 1: x's sibling w is red.



Handling: Switch colours of w and $x\,.\,p$ and perform a left rotation at $x\,.\,p$.

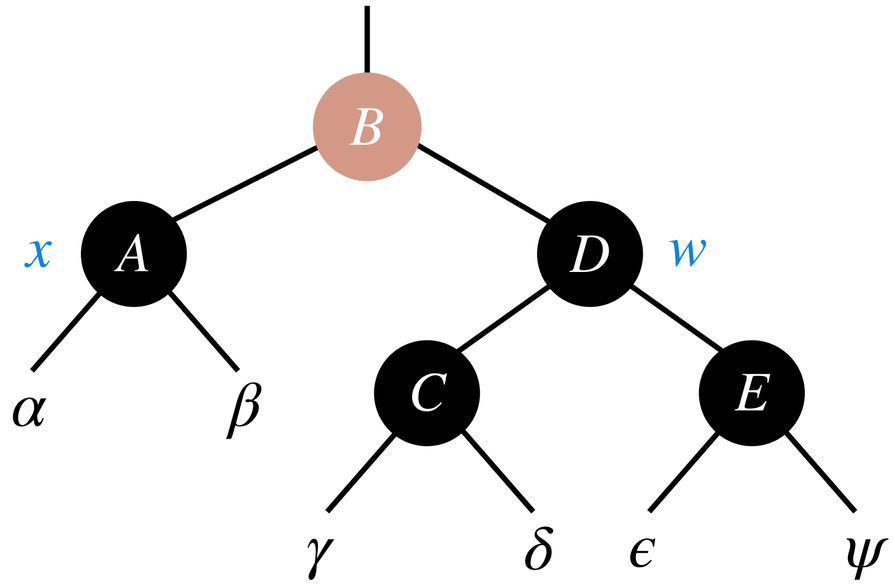
Cases for Deletion

Cases for Deletion

Case 2: x is NIL or x's sibling w is black, and both of w's children are **black**.

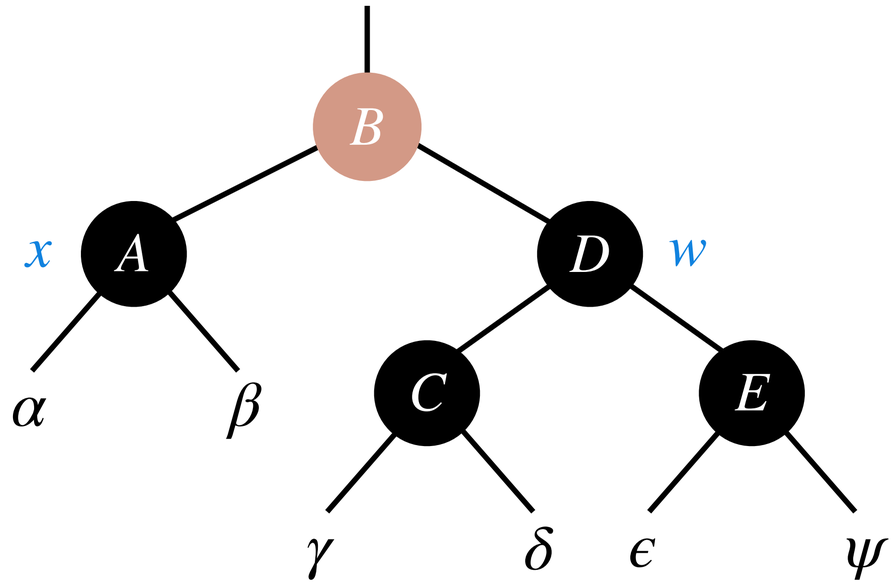
Cases for Deletion

Case 2: x is NIL or x's sibling w is black, and both of w's children are **black**.



Cases for Deletion

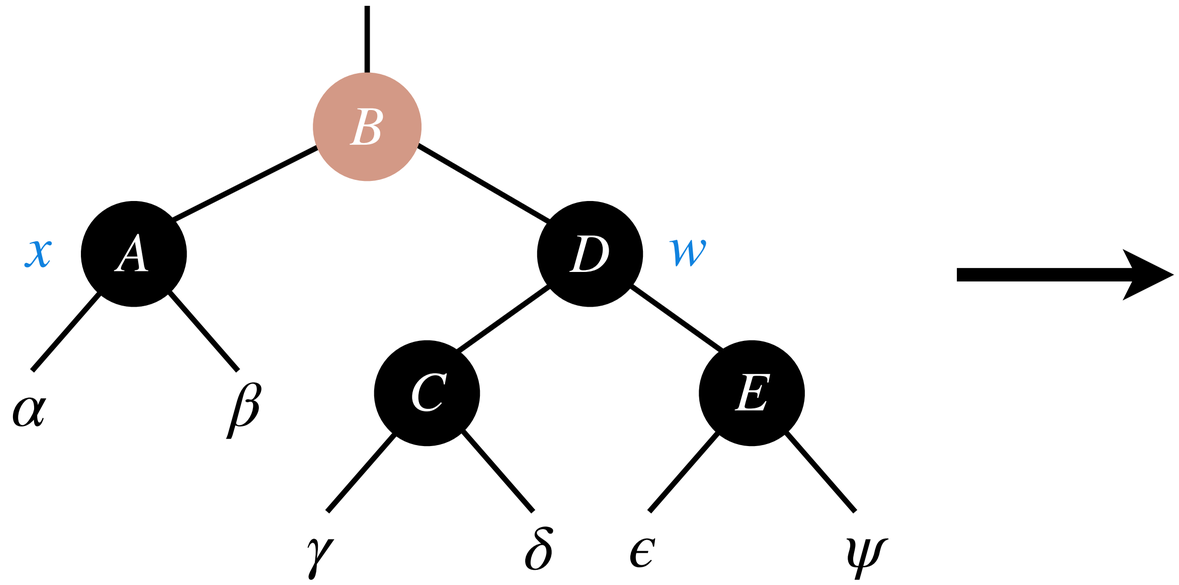
Case 2: x is NIL or x's sibling w is black, and both of w's children are **black**.



Handling: Move "extra blackness" of x and blackness of w to their **parent**.

Cases for Deletion

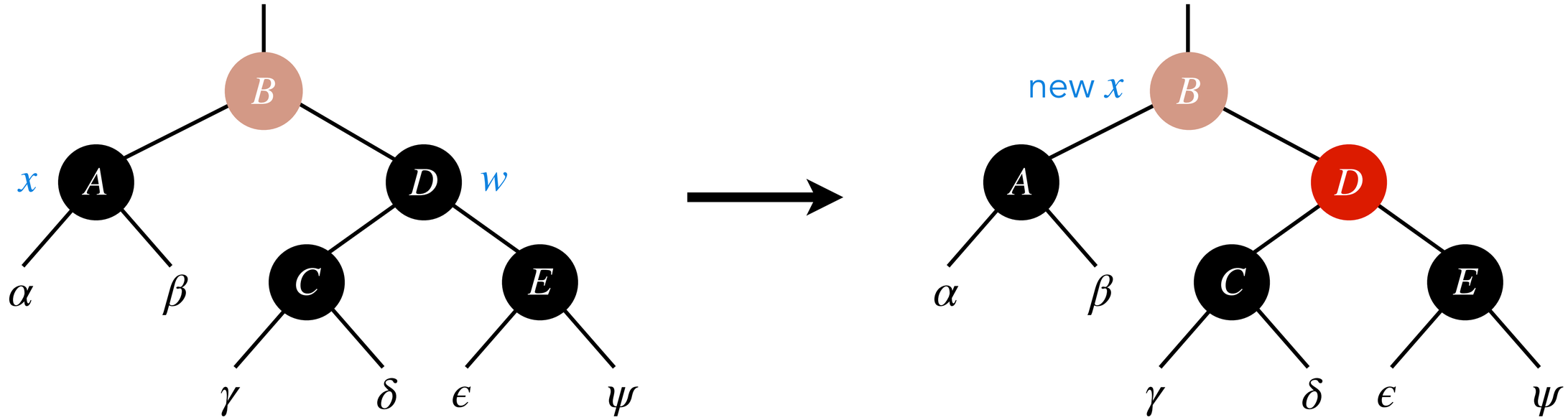
Case 2: x is NIL or x's sibling w is black, and both of w's children are **black**.



Handling: Move "extra blackness" of x and blackness of w to their **parent**.

Cases for Deletion

Case 2: x is NIL or x's sibling w is black, and both of w's children are **black**.



Handling: Move "extra blackness" of x and blackness of w to their **parent**.

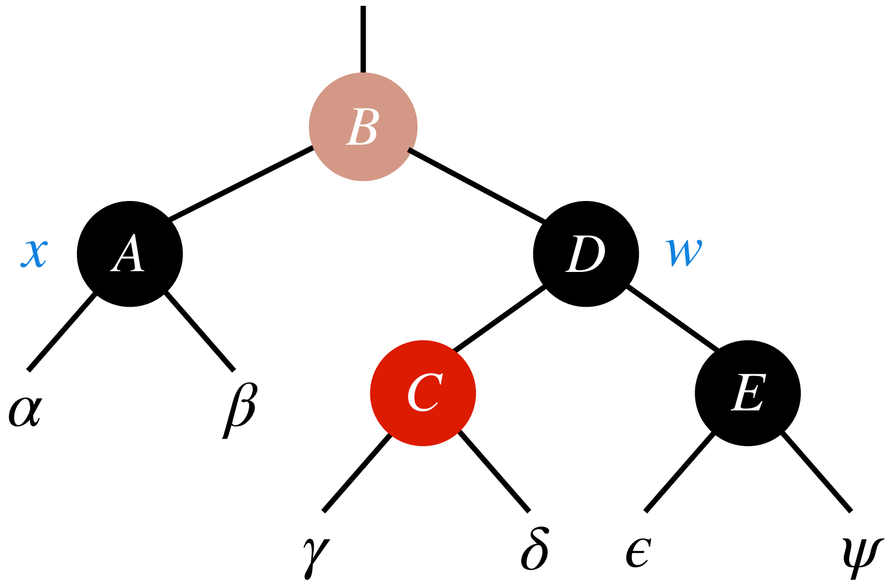
Cases for Deletion

Cases for Deletion

Case 3: x's sibling w is black, and w's left child is red, and w's right child is **black**.

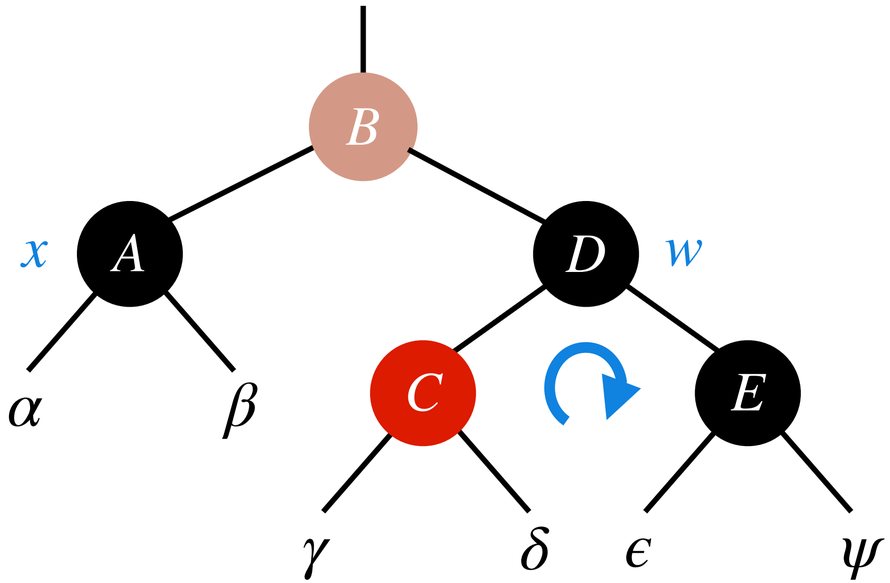
Cases for Deletion

Case 3: x's sibling w is black, and w's left child is red, and w's right child is **black**.



Cases for Deletion

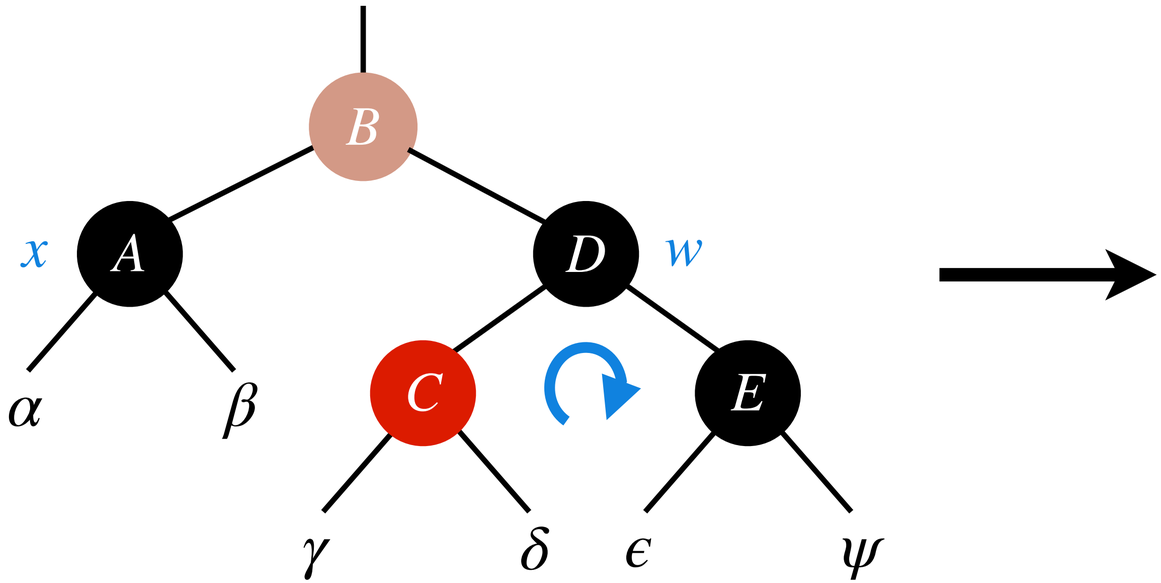
Case 3: x's sibling w is black, and w's left child is red, and w's right child is **black**.



Handling: Switch colour of w and its left child and then perform a right rotation on w.

Cases for Deletion

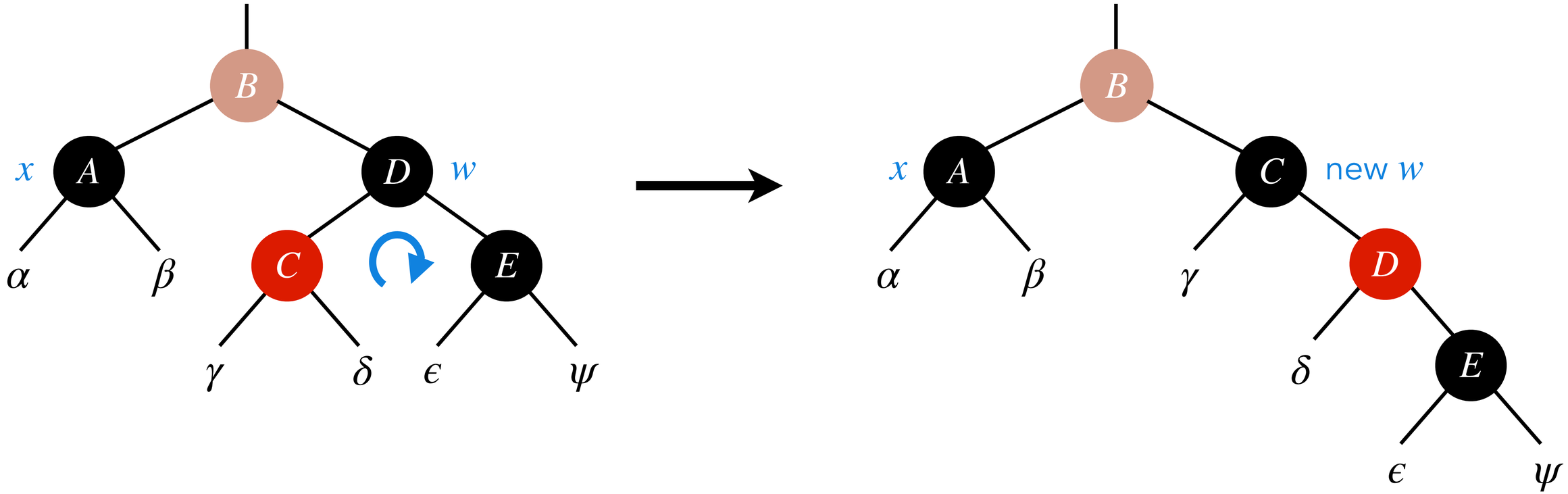
Case 3: x's sibling w is black, and w's left child is red, and w's right child is **black**.



Handling: Switch colour of w and its left child and then perform a right rotation on w.

Cases for Deletion

Case 3: x's sibling w is black, and w's left child is red, and w's right child is **black**.



Handling: Switch colour of w and its left child and then perform a right rotation on w.

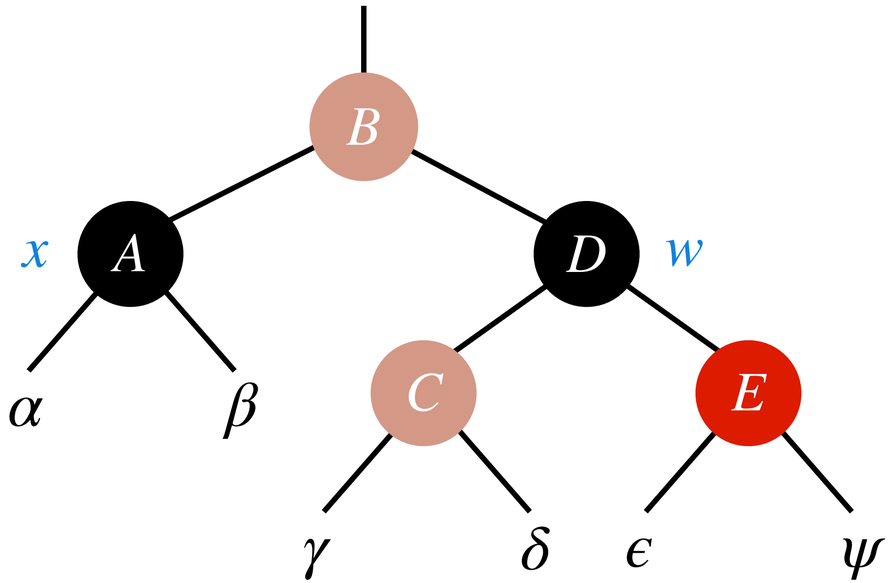
Cases for Deletion

Cases for Deletion

Case 4: x's sibling w is black, and w's right child is red.

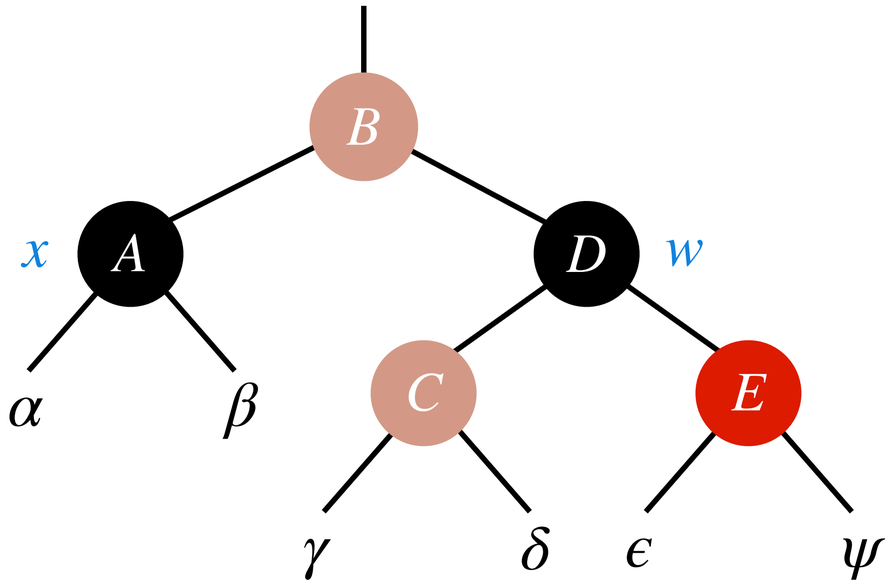
Cases for Deletion

Case 4: x's sibling w is black, and w's right child is red.



Cases for Deletion

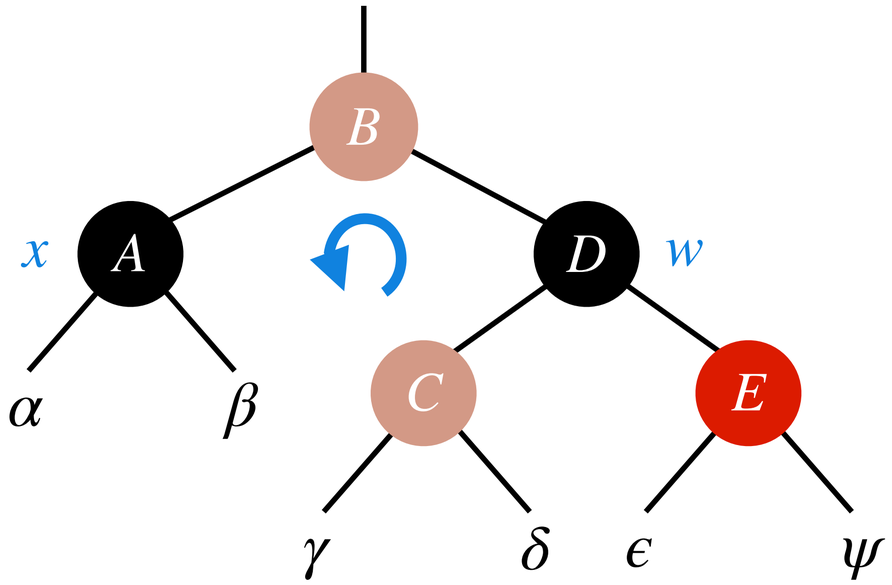
Case 4: x's sibling w is black, and w's right child is red.



Handling: Give w the colour of x's parent. Make x's parent and w's right child black.

Cases for Deletion

Case 4: x's sibling w is black, and w's right child is red.

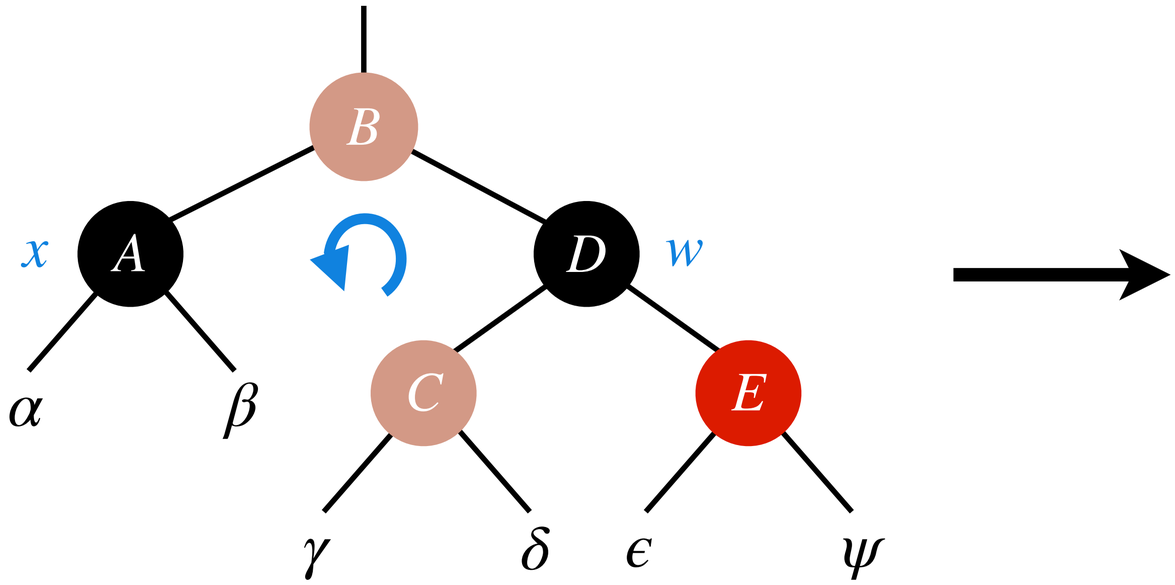


Handling: Give w the colour of x's parent. Make x's parent and w's right child black.

Perform left rotation at x's parent.

Cases for Deletion

Case 4: x's sibling w is black, and w's right child is red.

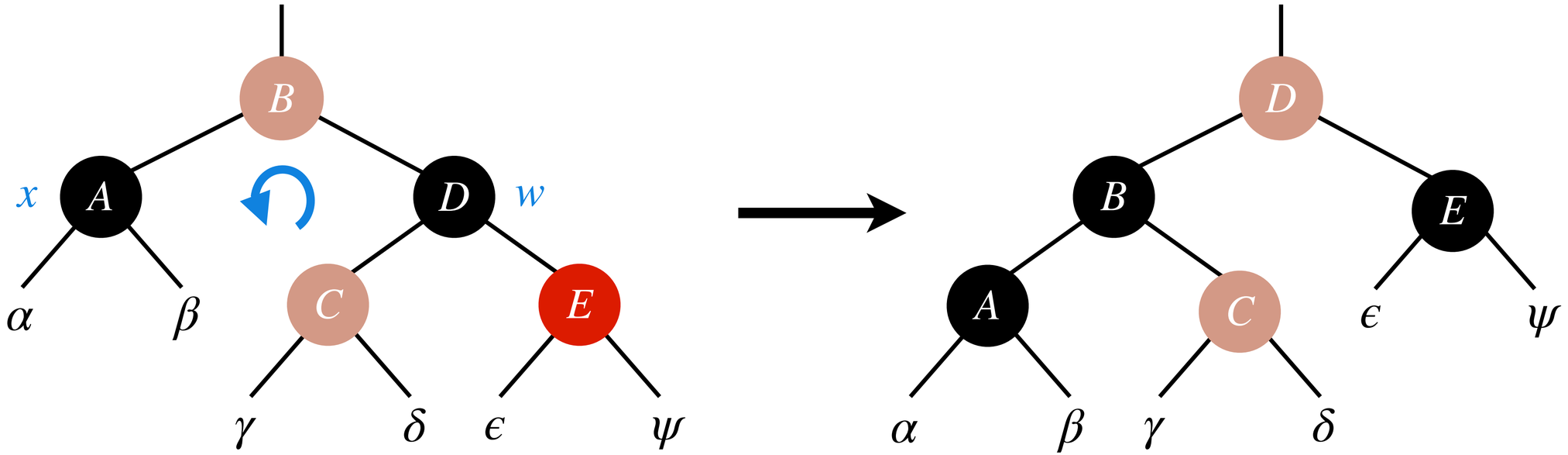


Handling: Give w the colour of x's parent. Make x's parent and w's right child black.

Perform left rotation at x's parent.

Cases for Deletion

Case 4: x's sibling w is black, and w's right child is red.

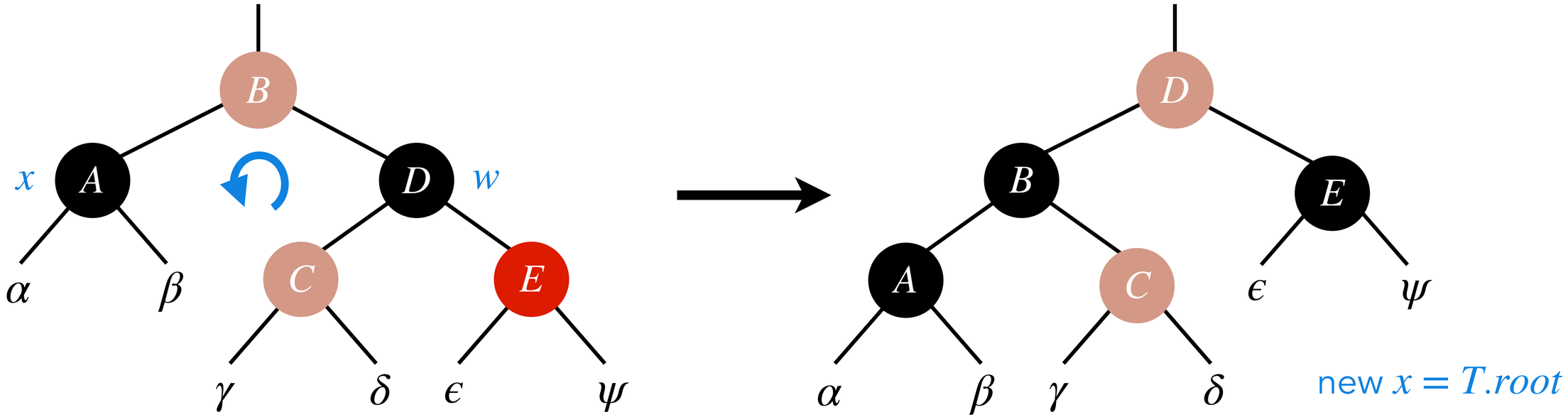


Handling: Give w the colour of x's parent. Make x's parent and w's right child black.

Perform left rotation at x's parent.

Cases for Deletion

Case 4: x's sibling w is black, and w's right child is red.



Handling: Give w the colour of x's parent. Make x's parent and w's right child black.

Perform left rotation at x's parent.

Facilitating New Operations

Facilitating New Operations

Defn: In a set, rank of an element is its position in the sorted order (w.r.t. keys) of the set.

Facilitating New Operations

Defn: In a set, rank of an element is its position in the sorted order (w.r.t. keys) of the set.

Example: Let $S = \{10,5,15,20,8,25,40,30\}$ be a set with elements containing only keys.

Facilitating New Operations

Defn: In a set, rank of an element is its position in the sorted order (w.r.t. keys) of the set.

Example: Let $S = \{10,5,15,20,8,25,40,30\}$ be a set with elements containing only keys.

Elements with ranks 1, 3, and 8 are 5, 10, and 40, respectively.

Facilitating New Operations

Defn: In a set, rank of an element is its position in the sorted order (w.r.t. keys) of the set.

Example: Let $S = \{10,5,15,20,8,25,40,30\}$ be a set with elements containing only keys.

Elements with ranks 1, 3, and 8 are 5, 10, and 40, respectively.

Ranks of elements 5, 15, and 25, are 1, 4, and 6, respectively.

Facilitating New Operations

Facilitating New Operations

Suppose we want to maintain a dynamic set with the following operations:

Facilitating New Operations

Contains elements with keys.

Suppose we want to maintain a dynamic set with the following operations:

Facilitating New Operations

Contains elements with keys.

Suppose we want to maintain a dynamic set with the following operations:

- Find the element of the ith rank.

Facilitating New Operations

Contains elements with keys.

Suppose we want to maintain a dynamic set with the following operations:

- Find the element of the ith rank.
- Given an element find its rank in the set.

Facilitating New Operations

Contains elements with keys.

Suppose we want to maintain a dynamic set with the following operations:

- Find the element of the ith rank.
- Given an element find its rank in the set.

Should we learn or invent a new data structure to facilitate these operations?

Facilitating New Operations

Contains elements with keys.

Suppose we want to maintain a dynamic set with the following operations:

- Find the element of the ith rank.
- Given an element find its rank in the set.

Should we learn or invent a new data structure to facilitate these operations?

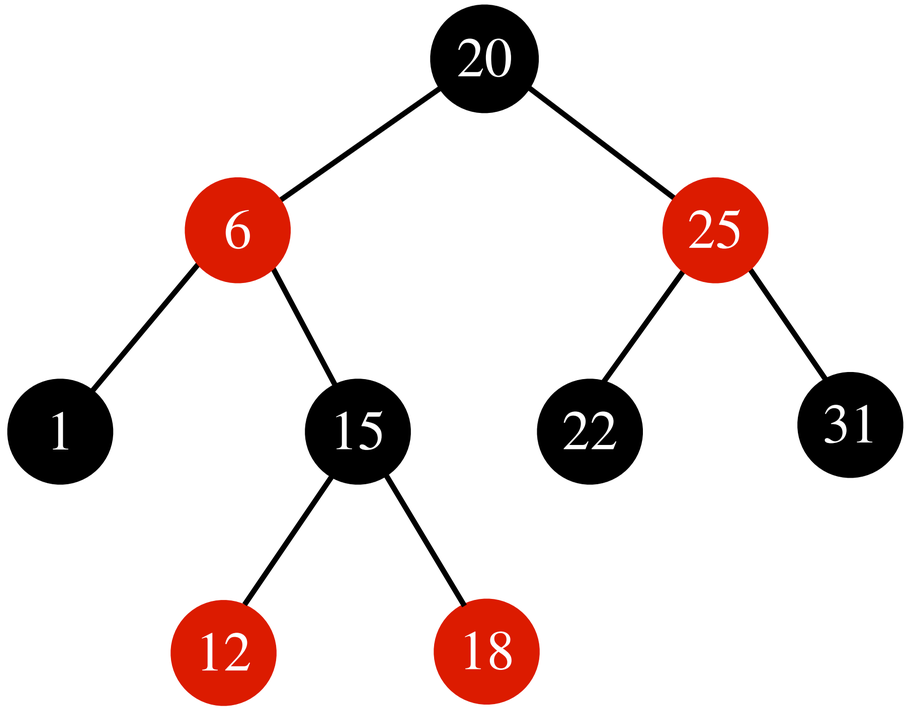
No. An easy modification to RB-trees is enough.

A Minor Modification to RB-Tree

Every node also stores the number of nodes in the subtree rooted at that node.

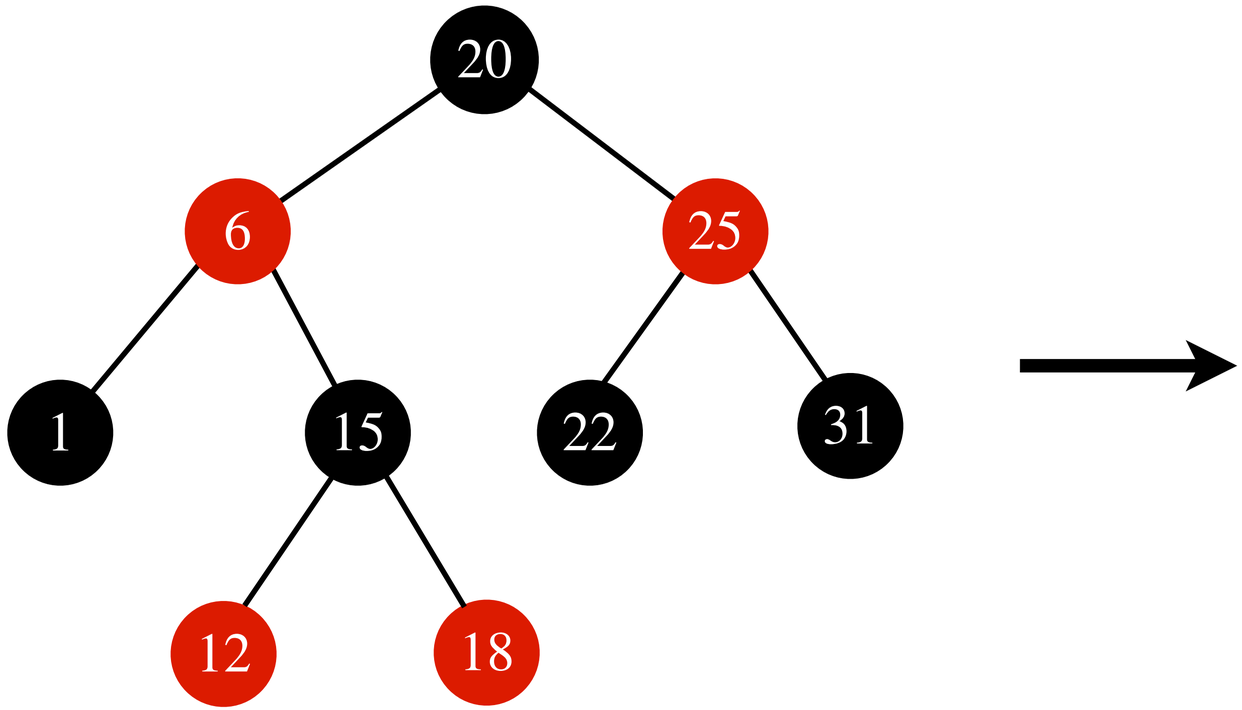
A Minor Modification to RB-Tree

Every node also stores the number of nodes in the subtree rooted at that node.



A Minor Modification to RB-Tree

Every node also stores the number of nodes in the subtree rooted at that node.



A Minor Modification to RB-Tree

Every node also stores the number of nodes in the subtree rooted at that node.